TECHNICAL DESIGN DOCUMENT

TravelPort

Architecture, Data Design & Integration Guide

VERSION	BACKEND	FRONTEND	DATABASE
v1.0.0	.NET 8 Clean Architecture	React 18 + TypeScript	SQL Server + EF Core 8

Table of Contents

Technical Design Document — TravelPort v1.0.0

1 Clean Architecture Overview

Layer responsibilities and dependency rules

1.1 Layer Responsibilities

API (Presentation) — Controllers, Middleware (JWT, exception handling, logging), Action Filters, Program.cs (DI root)

depends on ↓

Application (Business Logic) — AuthService, FlightService, HotelService, BookingService, WalletService, AdminService; DTOs, FluentValidation validators, ICacheService, IUnitOfWork

depends on ↓

Domain (Core) — Entities (User, Flight, Hotel, Booking, Wallet, WalletTransaction, Coupon, RefreshToken), Enums (BookingStatus, BookingType, TravelClass), ValueObjects — ZERO external dependencies

Infrastructure & Persistence implement interfaces defined in ↑

Persistence — TravelPortDbContext, Repositories (Flight, Hotel, Booking, Wallet, Coupon, User, Admin), EF Core Migrations, DataSeeder

Infrastructure — JwtService, CacheService, SmtpEmailService, RazorpayService, DuffelProvider; BackgroundServices (BookingExpiryWorker, RefreshTokenCleanupWorker)

1.2 Dependency Rule

```
Domain ← Application ← Infrastructure ← API
              ↑                  ↑
         Persistence        External APIs

# Arrows point INWARD. Outer layers know about inner layers.
# Inner layers (Domain) know nothing about outer layers.
# This is enforced via project references in .csproj files.
```

2 Backend Design

Entities, services, repositories and caching

2.1 Core Domain Entities

ENTITY	KEY FIELDS	NOTES
User	`Id, Email, PasswordHash, Role, IsBlocked, WalletId`	Wallet created on registration via cascade
Flight	`Id, FlightNumber, Origin, Destination, Airline, DepartureTime, ArrivalTime, AvailableSeats, BasePrice`	900+ seeded across 42 routes
Hotel	`Id, Name, City, StarRating, PricePerNight, Amenities, RoomTypes`	60+ across 12 cities
Booking	`Id, BookingReference (FL/HT prefix), UserId, FlightId?, HotelId?, FinalAmount, Status, BookingType`	Status: Pending → Confirmed / Cancelled
Wallet	`Id, UserId, Balance`	One wallet per user; balance validated before deduction
WalletTransaction	`Id, WalletId, Amount, Type (Credit/Debit), Description, ReferenceId`	Immutable audit trail
Coupon	`Id, Code, DiscountType (Percentage/Fixed), DiscountValue, MinOrderValue, IsActive, CouponType`	Flight-specific or Hotel-specific
RefreshToken	`Id, UserId, Token, ExpiresAt, IsRevoked`	Cleaned up by background worker every 24h

2.2 Caching Strategy

CACHE KEY PATTERN	TTL	INVALIDATED ON
`flights:{Origin}:{Destination}:{Date}:{Pax}:{Class}`	5 min	Flight booking (seat count change)
`flight:{id}`	30 min	Flight booking

CACHE KEY PATTERN	TTL	INVALIDATED ON
`hotels:{City}:{In}:{Out}:{Guests}:{Rating}:{Sort}`	5 min	Hotel booking
`hotel:{id}`	30 min	Hotel booking

`ICacheService` wraps `IDistributedCache` with JSON serialization. Default: `AddDistributedMemoryCache()`. Production swap: replace with `AddStackExchangeRedisCache()` in Program.cs — zero service-layer changes.

2.3 Atomic Booking + Wallet

```
// FlightService.BookAsync — simplified flow
1. ValidateFlightExists(flightId)
2. CheckSeatsAvailable(flight, passengerCount)
3. couponDiscount = ApplyCoupon(couponCode, baseAmount)  // optional
4. if (useWallet) _wallet.DeductAsync(userId, finalAmount)  // NO SaveChanges yet
5. booking = new Booking { ... Status = Pending }
6. _bookings.AddAsync(booking)
7. flight.AvailableSeats -= passengerCount
8. _unitOfWork.SaveChangesAsync()  // ← ONE atomic commit: booking + wallet + seats
9. _cache.RemoveAsync("flights:{key}")
10. _email.SendBookingConfirmed(booking)
```

3 Frontend Architecture

Feature modules, state management and API integration

3.1 Feature Module Structure

```
frontend/src/
├── api/
|   ├── axios.ts          ← Axios instance + JWT interceptor (auto-refresh)
|   └── endpoints.ts      ← All API URL constants (no hardcoded URLs elsewhere)
├── components/
|   ├── common/           ← Button, Select, Modal, Skeleton, ConfirmDialog
|   └── ui/               ← Design system primitives
├── features/
|   ├── auth/             ← authSlice (Redux), LoginForm, RegisterForm
|   ├── flights/          ← FilterSidebar, FlightCard, FarePopup, TravellerDetails
|   ├── hotels/           ← HotelCard, HotelFilters, RoomSelector
|   ├── bookings/         ← BookingCard, BookingDetailPage, PDFDownload
|   └── admin/            ← AdminPage (4 tabs: Dashboard, Users, Bookings, Coupons)
├── pages/                ← Route-level components (FlightsPage, HotelsPage, etc.)
├── services/             ← flightService, hotelService, bookingService, userService
├── store/                ← Redux store + authSlice
├── types/index.ts        ← All shared TypeScript interfaces
└── routes/               ← AppRouter, PrivateRoute, AdminRoute
```

3.2 JWT Interceptor Pattern

```
// api/axios.ts
api.interceptors.response.use(
  (res) => res,
  async (error) => {
    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      const token = await refreshAccessToken();  // POST /auth/refresh
      api.defaults.headers.common['Authorization'] = `Bearer ${token}`;
      return api(originalRequest);
    }
    return Promise.reject(error);
  }
);
```

3.3 Key Design Choices

DECISION	CHOICE	REASON
State management	`Redux Toolkit`	Predictable auth state; DevTools support; minimal boilerplate
Validation	`Zod`	Runtime type safety matching backend FluentValidation contracts
Styling	`Tailwind CSS v3`	Utility-first; no CSS-in-JS overhead; purged in production
Build tool	`Vite 6`	Fast HMR; native ESM; proxy config for /api → backend
HTTP	`Axios`	Interceptor support for JWT refresh; typed response wrapper
Named exports	`All components`	Avoids default-export aliasing confusion at import sites

4 Database Design

Core tables, relationships and migration strategy

4.1 Entity Relationships

RELATIONSHIP	CARDINALITY	FK
User → Wallet	1 : 1	`Wallet.UserId (Cascade delete)`
User → RefreshToken	1 : many	`RefreshToken.UserId`
User → Booking	1 : many	`Booking.UserId`
Wallet → WalletTransaction	1 : many	`WalletTransaction.WalletId`
Flight → Booking	1 : many (nullable)	`Booking.FlightId`
Hotel → Booking	1 : many (nullable)	`Booking.HotelId`
Hotel → RoomType	1 : many	`RoomType.HotelId`

4.2 Migration Strategy

EF Core Code-First migrations. Auto-applied on API startup via `db.Database.Migrate()` in `Program.cs`. DataSeeder runs after migration — idempotent checks prevent re-seeding on restart.

5 Authentication Flow

JWT + Refresh token rotation

1. **Login**

   POST /auth/login → validate credentials (BCrypt) → issue JWT (15 min) + RefreshToken (7 days, stored in DB)

2. **API Call**

   Client sends Authorization: Bearer {access_token} header

Axios interceptor catches 401 → POST /auth/refresh → new access token issued

4 **Token Rotation**

Old refresh token revoked; new pair issued. IsRevoked flag set in DB.

5 **Cleanup**

RefreshTokenCleanupWorker deletes revoked/expired tokens every 24 hours

7 External Integrations

RAZORPAY

Payment gateway. Enabled via config flag. Mock fallback in dev returns success.

SMTP / OFFICE365

Transactional email. HTML table layouts for Gmail/Outlook compatibility. TLS 587.

DUFFEL

Real flight search API. Disabled by default — rich DB seed data used instead.